

Lecture 1 计算机核心数学根基

周建宇

1 数学建模 (DAY 1)

问题 $\rightarrow$ 模型 $\rightarrow$ 算法 $\rightarrow$ 代码

1.1 KMP 算法

在字符串匹配问题中，主串（文本）记为 T ，其长度为 n ；模式串（待查找的子串）记为 P ，其长度为 m 。字符串 T 的第 i 个字符记为 $T[i]$ ，模式串 P 的第 j 个字符记为 $P[j]$ 。区间 $T[i..j]$ 表示从 $T[i]$ 到 $T[j]$ 的连续子串。

问题定义：给定主串 $T[1..n]$ 和模式串 $P[1..m]$ ，求所有起始位置 i ，使得

$$T[i..i+m-1] = P[1..m], \quad 1 \leq i \leq n-m+1.$$

这里的“匹配”表示模式串 P 的每个字符与对应主串子串的字符一一相等；“失配”表示当前比较的字符不相等。

这是一个典型的字符串匹配问题：从问题出发，我们先建立模式串的前缀函数模型，再用该模型指导匹配过程。

Definition 1.1.1. KMP 算法 (Knuth–Morris–Pratt 算法) 是一种字符串模式匹配算法。它通过预处理模式串得到前缀函数 π ，记录每个位置最长相等真前缀和真后缀长度，从而在失配时无需回退主串指针，实现 $O(n+m)$ 时间匹配。

Definition 1.1.2 (真前缀与真后缀). 给定字符串 $P[1..q]$:

- **真前缀 (proper prefix)**: P 的前缀但不包含整个字符串本身，即形如 $P[1..k]$ 且 $0 \leq k < q$ 的子串；
- **真后缀 (proper suffix)**: P 的后缀但不包含整个字符串本身，即形如 $P[q-k+1..q]$ 且 $0 \leq k < q$ 的子串。

Definition 1.1.3 (前缀函数). 模式串 $P[1..m]$ 的前缀函数 $\pi[1..m]$ 定义为: 每个位置 q 处的值等于 $P[1..q]$ 的最长相等真前缀与真后缀的长度, 即

$$\pi[1] = 0,$$

$$\pi[q] = \max\{k \mid 0 \leq k < q, P[1..k] = P[q - k + 1..q]\}, \quad 2 \leq q \leq m.$$

例如, 令模式串

$$P = \text{ababaca}, \quad m = 7.$$

则各位置的前缀函数为:

$$\pi = [0, 0, 1, 2, 3, 0, 1],$$

其中:

- $\pi[3] = 1$, 因为 $P[1..1] = \text{a}$ 与 $P[3..3] = \text{a}$, 且没有更长的真前缀与真后缀匹配;
- $\pi[5] = 3$, 因为 $P[1..3] = \text{aba}$ 与 $P[3..5] = \text{aba}$;
- $\pi[6] = 0$, 因为 $P[1] \neq P[6]$, 此时没有非空真前缀与真后缀相同;
- $\pi[7] = 1$, 因为 $P[1..1] = \text{a}$ 与 $P[7..7] = \text{a}$ 。

匹配过程: 令主串 $T[1..n]$, 模式串 $P[1..m]$, 当前已匹配长度 $q = 0$ 。遍历 $i = 1 \dots n$ 表示按顺序扫描主串 T 的第 i 个字符 $T[i]$, 在每一步将它与模式串中下一个待比较字符 $P[q + 1]$ 进行比较。若失配, 则

$$q \leftarrow \pi[q - 1]$$

直到重新匹配或退到 0。

例如, 令主串和模式串为:

$$T = \text{abababaca}, \quad P = \text{ababaca}.$$

已知 $\pi = [0, 0, 1, 2, 3, 0, 1]$ 。前 6 步匹配过程如下:

- $i = 1$: $T[1] = \text{a}$ 与 $P[1] = \text{a}$ 匹配, $q = 1$;
- $i = 2$: $T[2] = \text{b}$ 与 $P[2] = \text{b}$ 匹配, $q = 2$;

- $i = 3$: $T[3] = a$ 与 $P[3] = a$ 匹配, $q = 3$;
- $i = 4$: $T[4] = b$ 与 $P[4] = b$ 匹配, $q = 4$;
- $i = 5$: $T[5] = a$ 与 $P[5] = a$ 匹配, $q = 5$;
- $i = 6$: $T[6] = b$ 与 $P[6] = c$ 失配, 于是 $q \leftarrow \pi[5] = 3$, 接着比较 $P[4] = b$ 与 $T[6] = b$, 继续匹配, 得到 $q = 4$;
- $i = 7$: $T[7] = a$ 与 $P[5] = a$ 匹配, $q = 5$;
- $i = 8$: $T[8] = c$ 与 $P[6] = c$ 匹配, $q = 6$;
- $i = 9$: $T[9] = a$ 与 $P[7] = a$ 匹配, $q = 7 = m$, 找到一个完整匹配, 起始位置为 $i - m + 1 = 3$ 。

这样, KMP 利用前缀函数避免了回退主串指针, 直接从模式串已知的最长可重用前缀长度继续比较。

计算前缀函数和查找匹配的伪代码如下:

```
function compute_prefix(P):
    m = len(P)
    pi = array[1..m]
    pi[1] = 0
    k = 0
    for q = 2 to m:
        while k > 0 and P[k+1] != P[q]:
            k = pi[k]
        if P[k+1] == P[q]:
            k = k + 1
        pi[q] = k
    return pi

function kmp_search(T, P):
    n = len(T)
    m = len(P)
    pi = compute_prefix(P)
```

```

q = 0
results = []
for i = 1 to n:
    while q > 0 and P[q+1] != T[i]:
        q = pi[q]
    if P[q+1] == T[i]:
        q = q + 1
    if q == m:
        results.append(i - m)
        q = pi[q]
return results

```

Note 1.1.1. 与 KMP 相关的字符串进阶数据结构与算法：

- **Aho-Corasick 自动机**：多模式串匹配算法。把全部模式串建成 Trie 树，再仿照 KMP 的前缀函数为每个节点建立失配指针（fail 指针），扫描主串一次即可同时匹配所有模式串，总复杂度 $O(n + m + \text{匹配数})$ （ m 为所有模式串长度之和），常用于敏感词过滤、多关键字检索。
- **后缀数组 (Suffix array, SA)**：把字符串的所有后缀按字典序排序得到的数组，配合最长公共前缀数组 (LCP 数组) 可高效解决子串查找、最长重复子串、不同子串计数等问题，倍增法构建复杂度 $O(n \log n)$ 。
- **后缀自动机 (Suffix automaton, SAM)**：能识别一个字符串的所有子串的最小确定性有限自动机，状态数与转移数均为 $O(n)$ ，可以线性时间在线构造，用于统计子串出现次数、求最长公共子串等。
- **后缀树 (Suffix tree)**：把所有后缀压缩存储的字典树， $O(n)$ 个节点，可在 $O(|P|)$ 时间内判断模式串 P 是否为主串的子串。
- **滚动哈希 (Rolling hash)**：把字符串视作一个进制数并取模，用 $O(1)$ 时间从上一窗口的哈希值推出相邻窗口的哈希值，从而快速比较子串是否相等；理论上有碰撞风险，实践中可用双哈希降低碰撞概率。
- **倒排索引 (Inverted index)**：从词项映射到包含该词的文档列表的索引结构（词典 $\rightarrow$ 倒排表），与“文档 $\rightarrow$ 词项”的正排索引相对，是搜索引擎全文检索的核心数据结构。

- **Shingling**: 将文档切分为连续的 k 个 token 组成的集合 (k -gram), 用集合的 Jaccard 相似度度量两份文档的相似程度。
- **MinHash**: 对集合元素施加多种随机哈希, 取每组哈希值的最小值作为文档签名, 使得签名相等的概率恰好等于集合的 Jaccard 相似度, 从而在大规模语料上近似估计文档相似度 (网页去重)。
- **SimHash**: 把文本中每个特征词哈希为二进制向量, 按权重累加后取符号得到定长指纹, 两个指纹的汉明距离小则认为文档相似; 适合海量网页去重。

Definition 1.1.4 (Dijkstra 算法). 求解非负权图单源最短路径的经典算法。维护集合 S (已确定最短距离的顶点) 和距离数组 d , 每轮从未确定顶点中选出 d 最小的顶点 u 加入 S , 并对其所有出边做松弛操作; 用二叉堆优化后复杂度为 $O((n + m) \log n)$ 。

Definition 1.1.5 (松弛操作 (relaxation)). 对边 (u, v) , 若

$$d[v] > d[u] + w(u, v),$$

则更新 $d[v] \leftarrow d[u] + w(u, v)$ 。松弛是 Dijkstra、Bellman-Ford、SPFA 等最短路径算法共用的核心步骤: 用一条”可能更短的路”去改善当前已知的最短距离估计。

Dijkstra 算法的伪代码如下 (PQ 为小根堆, 键为距离):

```
function dijkstra(s):
    for v in V: d[v] = INF
    d[s] = 0
    PQ = min-heap containing (0, s)
    while PQ not empty:
        (du, u) = PQ.extract_min()
        if du != d[u]: continue          // 过期条目
        for each edge (u, v, w):
            if d[v] > d[u] + w:          // 松弛操作
                d[v] = d[u] + w
                PQ.insert((d[v], v))
    return d
```

$$\text{问题} \begin{cases} \text{生活问题} \\ \text{科学问题} \\ \text{可建模的科学问题} \end{cases} \begin{cases} \text{假设} \\ \text{规模} \end{cases}$$

Note 1.1.2. 作业：报告，可 AI 生成，回顾本科所学的知识，有哪些知识，经过这五天的学习，哪些知识有了更深入的理解，前因后果，核心理解。

Example 1.1.1. 计算机如何计算 $\binom{m}{n} = \frac{m!}{n!(m-n)!}$? $= \frac{\prod_{i=1}^n (m-i+1)}{\prod_{i=1}^n i} = \prod_{i=1}^n \frac{m-i+1}{i}$, n 个连续自然数一定会被 n 整除, or 利用组合数公式 $\binom{m}{n} = \binom{m-1}{n-1} + \binom{m-1}{n}$ 。

思路：直接计算 $m!/(n!(m-n)!)$ 在 m 较大时很快溢出；改按 $\prod_{i=1}^n \frac{m-i+1}{i}$ 逐步相乘可避免中间值爆炸（每一步结果都是整数，因为 n 个连续自然数必含 n 的因子）。也可以按帕斯卡递推 $\binom{m}{n} = \binom{m-1}{n-1} + \binom{m-1}{n}$ 用 $O(nm)$ 的动态规划填表，且中间值保持在 $\binom{m}{n}$ 同量级内，适合精确大数计算。

1.2 等价变化

Example 1.2.1. m 个完全的小球放进 n 个完全相同的盘子里？

$$F_{m,n} = \begin{cases} 1, & n = 1 \text{ 或 } m = 0 \\ F_{m,m} & n > m \\ F_{m,n-1} (\text{有空盘}) + F_{m-n,n} (\text{无空盘, 每个盘子至少有一个小球}), & n > 1 \text{ 且 } m > 0 \end{cases}$$

思路：由于球和盘都不可区分，只需关心“每盘装几个”的非增整数分拆（partition）。递推的关键是枚举是否允许空盘：若至少有一个空盘，则去掉一个盘子化为 $F_{m,n-1}$ ；若没有空盘，则先从每个盘子拿掉一个小球，化为 $F_{m-n,n}$ 。边界 $n > m$ 时空盘不可避免，等价于 $F_{m,m}$ 。这是“整数分拆数”模型的等价变化。

Example 1.2.2 (错位排序). n 个盒子有不同标签 $1, 2, \dots, n$, n 个不同的小球 $1, 2, \dots, n$, 有不同标签，放进盒子里，每个盒子有一个小球，问有多少种放法？ $n!$ 种放法。若要求盒子的标签与小球的标签不同，问有多少种放法？设 $f(n)$ 为所求的放法数，则有递推关系：

$$f(n) = (n-1)(f(n-1) + f(n-2)), \quad n \geq 3$$

思路（容斥 / 第一球落位分析）：错位排列的经典证明：考虑小球 n ，它可放在 $n-1$ 个非

盒子 n 的位置之一。若小球 n 放在盒子 k 且小球 k 放在盒子 n ，则剩下 $n-2$ 个球全错位，共 $f(n-2)$ 种；若小球 k 不放盒子 n ，则把“盒子 n 必须避开小球 k ”纳入错位要求，等价于 $n-1$ 个元素的错位排列 $f(n-1)$ 种。于是 $f(n) = (n-1)(f(n-1) + f(n-2))$ 。也可用容斥原理 $f(n) = n! \sum_{k=0}^n \frac{(-1)^k}{k!}$ 。

Example 1.2.3. n 个节点的二叉树有多少种？设 $f(n)$ 为所求的二叉树数，则有递推关系：

$$f(n) = \sum_{i=0}^{n-1} f(i)f(n-i-1), \quad n \geq 1$$

思路：按根节点划分左右子树。固定根后，左子树含 i 个节点、右子树含 $n-i-1$ 个节点，左右独立，故对 i 求和得卷积递推 $f(n) = \sum_{i=0}^{n-1} f(i)f(n-i-1)$ ，即卡特兰数的递推定义，见下。

Note 1.2.1. 该递推给出的是第 n 个卡特兰数 (Catalan number) C_n (记 $f(n) = C_n$)。闭式为

$$C_n = \frac{1}{n+1} \binom{2n}{n}, \quad C_0 = 1.$$

生成函数为

$$C(x) = \sum_{n \geq 0} C_n x^n = \frac{1 - \sqrt{1-4x}}{2x}.$$

渐近行为为 $C_n \sim \frac{4^n}{\sqrt{\pi} n^{3/2}}$ 。卡特兰数计数多种组合结构，例如二叉树、合法括号序列、非交错路径等。

Definition 1.2.1 (Floyd-Warshall 算法). 求解全源最短路径的动态规划算法。设 $d_{ij}^{(k)}$ 为只允许经过 $\{1, \dots, k\}$ 作为中间点时 i 到 j 的最短路，递推式

$$d_{ij}^{(k)} = \min\{d_{ij}^{(k-1)}, d_{ik}^{(k-1)} + d_{kj}^{(k-1)}\},$$

复杂度 $O(n^3)$ ，允许负权边但不允许负环，常用于传递闭包等图论问题。伪代码如下：

```
function floyd_warshall(W):           // W 为边权矩阵
    d = W
    for k = 1 to n:
        for i = 1 to n:
            for j = 1 to n:
                d[i][j] = min(d[i][j], d[i][k] + d[k][j])
```

```
return d
```

Definition 1.2.2 (Bellman-Ford 算法). 求解单源最短路径, 可处理负权边。对全部边执行 $n-1$ 轮松弛, 第 k 轮结束后得到的是经过至多 k 条边的最短路; 若第 n 轮仍能松弛, 则存在负环。复杂度 $O(nm)$ 。伪代码如下:

```
function bellman_ford(s):
    for v in V: d[v] = INF
    d[s] = 0
    for k = 1 to n-1:
        for each edge (u, v, w):
            if d[v] > d[u] + w: d[v] = d[u] + w    // 松弛
    for each edge (u, v, w):                        // 检测负环
        if d[v] > d[u] + w: return "存在负环"
    return d
```

Definition 1.2.3 (SPFA). Bellman-Ford 的队列优化版本, 只对“距离被更新过的顶点”松弛其出边, 通常很快但最坏仍为 $O(nm)$, 也可用于检测负环。

Definition 1.2.4 (维特比算法 (Viterbi algorithm)). 在隐马尔可夫模型 (HMM) 中求解最可能隐状态序列的动态规划算法, 即给定观测序列求使后验概率最大的状态路径; 复杂度 $O(TN^2)$ (T 为序列长度, N 为状态数), 广泛用于词性标注、语音识别、纠错解码等。

Definition 1.2.5 (逆序对的排列). 若排列 $\pi = (\pi_1, \dots, \pi_n)$ 中存在 $i < j$ 且 $\pi_i > \pi_j$, 则 (π_i, π_j) 是一个逆序对; 逆序对数刻画排列的“乱序程度”, 并与冒泡排序的交换次数、排列的奇偶性密切相关。

1.3 重要假设

Example 1.3.1. 贝特朗悖论 (Bertrand paradox): 单位圆上任取一条弦, 问弦长 $d \geq \sqrt{3}$ (即不小于内接正三角形边长) 的概率是多少? 因为“随机取弦”的等可能性定义方式不同, 会得到不同答案:

- $\frac{1}{3}$: 随机取两个端点 (端点在圆周上均匀分布) —— 对应圆心角之差在 $[120^\circ, 240^\circ]$ 或等价的角范围;

- $\frac{1}{2}$: 随机取弦的中点沿某条直径均匀分布——中点必须落在距圆心 $\leq \frac{1}{2}$ 的范围内;
- $\frac{1}{4}$: 随机取弦的中点在整个圆内均匀分布——中点必须落在半径为 $\frac{1}{2}$ 的同心圆内。

该悖论说明：同一几何问题在”均匀”的不同理解下概率可以不同，数学建模必须先明确假设。

Definition 1.3.1 (K-means 算法). 给定 n 个样本和簇数 k ，把样本划分到 k 个簇中，使簇内平方和

$$\sum_{i=1}^k \sum_{x \in C_i} \|x - \mu_i\|^2$$

最小（即最小化类内距离，等价于间接最大化类间距离）。交替进行”分配：每个点归到最近的质心”与”更新：重新计算质心”两步直至收敛；是 NP-hard 问题的启发式求解，复杂度 $O(nkt)$ ，对初始质心和 k 的选择敏感。伪代码如下：

```
function kmeans(X, k):
    mu = random_k_centroids(X)
    repeat until centroids stop changing:
        C = empty assignment for each x
        for each x in X:
            assign x to cluster of nearest centroid mu
        for i = 1 to k:
            mu[i] = mean of all points in cluster i
    return clusters
```

Definition 1.3.2 (距离度量). 距离度量的选择影响聚类结果：常用欧氏距离 $\|x - y\|_2$ 、曼哈顿距离 $\|x - y\|_1$ 、余弦相似度 $\frac{x \cdot y}{\|x\| \|y\|}$ 等，距离定义本身就是建模中的关键假设。

Definition 1.3.3 (层次聚类). 不预设簇数，自底向上（凝聚式）每次合并距离最近的两个簇，或自顶向下（分裂式）每次拆分，最终得到一棵聚类树（树状图），可在任意高度切割得到不同粒度的簇。凝聚式层次聚类的伪代码如下：

```
function agglomerative_clustering(X):
    each point is its own cluster
    while more than one cluster remains:
```

```

merge the two clusters closest under linkage
record merge in dendrogram
return dendrogram

```

Definition 1.3.4 (DBSCAN). 基于密度的聚类算法，不需预设簇数。给定邻域半径 ε 和最小点数 MinPts：若某点 ε -邻域内点数 $\geq$ MinPts 则为核心点；核心点及其密度可达的点构成一个簇，其余点记为噪声。能发现任意形状的簇并识别离群点，配合空间索引复杂度约 $O(n \log n)$ 。伪代码如下：

```

function dbscan(X, eps, MinPts):
    label each point as unvisited
    for each unvisited point p:
        N = points within eps of p
        if |N| < MinPts: mark p as noise
        else:
            start new cluster C with p
            expand C to all density-reachable points
    return clusters

```

2 计算机硬件与问题规模 (DAY 2)

2.1 问题规模

Example 2.1.1. 算法：基于比较的排序。给定 n 个元素，比较类排序算法的最坏时间复杂度下界为 $\Omega(n \log n)$ （基于决策树）。常见排序算法及复杂度：

- **冒泡排序**：反复扫描序列，相邻逆序元素就交换，每趟把当前最大元素”冒泡”到末尾； $O(n^2)$ ，最好情况 $O(n)$ 。
- **插入排序**：将每个元素插入到已排序前缀的正确位置； $O(n^2)$ ，对近似有序序列接近 $O(n)$ ，稳定。

- **快速排序**：分治 + 划分，选枢轴把序列分成左右两部分递归排序；平均 $O(n \log n)$ ，最坏 $O(n^2)$ （见下），原地排序但递归栈占 $O(\log n)$ 空间，不稳定。
- **归并排序**：分治，先递归排序左右两半再线性归并；稳定， $O(n \log n)$ ，但需要 $O(n)$ 额外空间。归并排序的合并阶段伪代码如下：

```
function merge_sort(A, l, r):
    if l >= r: return
    mid = (l + r) / 2
    merge_sort(A, l, mid); merge_sort(A, mid+1, r)
    merge two sorted halves in  $O(r-l+1)$ 
```

Theorem 2.1.1 (比较排序下界). 任何基于比较的排序算法的最坏时间复杂度至少为

$$\Omega(n \log n),$$

精确地， $T(n) \geq \lceil \log_2(n!) \rceil$ 。

证明. 每次比较把可能的排列按结果分为两支，形成一棵决策树；排序算法必须能区分全部 $n!$ 种排列，故决策树的叶子数 $\geq n!$ ，高度至少 $\lceil \log_2(n!) \rceil$ 。由 Stirling 公式 $n! \sim \sqrt{2\pi n}(n/e)^n$ 得 $\log_2(n!) = \Theta(n \log n)$ 。□

Theorem 2.1.2 (主定理 (Master Theorem)) . 设递归式

$$T(n) = aT(n/b) + f(n),$$

其中 $a \geq 1$, $b > 1$ 为常数，且 $f(n)$ 为渐近正函数。则：

- 若 $f(n) = O(n^{\log_b a - \varepsilon})$ ，其中 $\varepsilon > 0$ 为任意小的常数，则 $T(n) = \Theta(n^{\log_b a})$ ；
- 若 $f(n) = \Theta(n^{\log_b a} \log^k n)$ ，其中 $k \geq 0$ ，则 $T(n) = \Theta(n^{\log_b a} \log^{k+1} n)$ ；
- 若 $f(n) = \Omega(n^{\log_b a + \varepsilon})$ ，其中 $\varepsilon > 0$ ，且满足正则条件 $af(n/b) \leq cf(n)$ （对足够大的 n ），则 $T(n) = \Theta(f(n))$ 。

主定理证明思路（递归树）。把递归式 $T(n) = aT(n/b) + f(n)$ 展开为递归树：树高 $h = \log_b n$ ，第 j 层的代价为 $a^j f(n/b^j)$ ，叶节点数为 $a^h = n^{\log_b a}$ ，每个叶节点代价 $O(1)$ 。总代价

$$T(n) = \sum_{j=0}^{\log_b n - 1} a^j f(n/b^j) + \Theta(n^{\log_b a}).$$

- 情形 1: $f(n) = O(n^{\log_b a - \varepsilon})$ 时，各层代价按几何级数递减，和由叶节点主导，故 $T(n) = \Theta(n^{\log_b a})$;
- 情形 2: $f(n) = \Theta(n^{\log_b a} \log^k n)$ 时，每层代价近似相等，共 $\log_b n$ 层，故 $T(n) = \Theta(n^{\log_b a} \log^{k+1} n)$;
- 情形 3: $f(n) = \Omega(n^{\log_b a + \varepsilon})$ 且正则条件保证非叶层代价递增，和由根主导，故 $T(n) = \Theta(f(n))$ 。

□

Note 2.1.1. 给出一个“不开辟额外空间”的快速排序实现思路：

```
def quick_sort(arr, left, right):
    if left >= right:
        return
    pivot = arr[right]
    i = left - 1
    for j in range(left, right):
        if arr[j] <= pivot:
            i += 1
            arr[i], arr[j] = arr[j], arr[i]
    arr[i + 1], arr[right] = arr[right], arr[i + 1]
    mid = i + 1
    quick_sort(arr, left, mid - 1)
    quick_sort(arr, mid + 1, right)
```

Note 2.1.2. 快速排序的平均时间复杂度（均匀性假设下）为 $O(n \log n)$ ，最坏情况为 $O(n^2)$ 。若每次划分都把序列分成 1 和 $n - 1$ 两部分，就会退化到 $O(n^2)$ 。例如，若每次都选到最

小值或最大值作为枢轴。对于 n 个互异元素，若每一层都只有“选最大”或“选最小”这两种选择，那么总共有 2^{n-1} 种最坏情况。

Definition 2.1.1 (群 (group)). 集合 G 连同二元运算 $\circ$ ，满足**封闭性** ($a \circ b \in G$)、**结合律** ($(a \circ b) \circ c = a \circ (b \circ c)$)、存在**单位元** e ($\forall a \in G, a \circ e = e \circ a = a$)、每个元素有**逆元** ($\forall a \in G, \exists a^{-1}, a \circ a^{-1} = a^{-1} \circ a = e$)。若还满足交换律则称为**阿贝尔群**。例：整数加法群 $(\mathbb{Z}, +)$ 。

Definition 2.1.2 (环 (ring)). 集合 R 上定义加法和乘法： $(R, +)$ 构成阿贝尔群，乘法满足结合律，且乘法对加法满足分配律。例：整数环 $\mathbb{Z}$ 。

Definition 2.1.3 (域 (field)). 交换环中若每个非零元都有乘法逆元（即 $(R \setminus \{0\}, \times)$ 构成阿贝尔群），则称为域。例： $\mathbb{Q}, \mathbb{R}, \mathbb{C}$ 及有限域 $\mathbb{F}_p$ 。

Definition 2.1.4 (格 (lattice)). 偏序集中任意两个元素都有最小上界（并）和最大下界（交）的结构；也可指数论/密码学中的 $\mathbb{R}^n$ 的离散加法子群（如 $\mathbb{Z}^n$ ），是格基密码的基础。

Definition 2.1.5 (确定性算法). 给定相同输入，执行的每一步都完全确定，输出必然相同。

Definition 2.1.6 (随机算法 (randomized algorithm)). 执行过程中利用随机性作决策的算法。两类典型：

- **拉斯维加斯算法 (Las Vegas algorithm)**：输出一定正确，但运行时间是一个随机变量（可能一直跑下去但概率为 0），如随机化快速排序；
- **蒙特卡洛算法 (Monte Carlo algorithm)**：运行时间确定，但输出可能以一定概率出错，可通过重复运行降低错误概率，如 MinHash、Miller-Rabin 素性测试。

例如对快速排序先做随机打乱 (shuffle)，即可避免固定有序输入导致的 $O(n^2)$ 最坏退化。

Note 2.1.3. 怎样改进快速排序算法，产生 $n!$ 种确定性算法？shuffle，随机打乱输入序列。

Note 2.1.4. 如何查找 n 个元素中第 k 小的元素？

一种思路是使用堆排序：先构建一个最大堆，然后依次取出堆顶元素。建堆的时间复杂度为 $O(n)$ ，每次取出堆顶并调整堆的时间复杂度为 $O(\log n)$ ，因此取出 k 次的总复杂度为 $O(n + k \log n)$ 。

另一种更高效的思路是快速选择算法。其核心思想与快速排序类似：通过一次划分把数组分成两部分。若枢轴位置恰好是 k ，则直接返回；若 k 小于枢轴位置，则只递归处理左边；若 k 大于枢轴位置，则只递归处理右边。其伪代码如下：

```

function quick_select(arr, left, right, k):
    p = partition(arr, left, right)
    rank = p - left + 1

    if rank == k:
        return arr[p]
    else if rank > k:
        return quick_select(arr, left, p - 1, k)
    else:
        return quick_select(arr, p + 1, right, k - rank)

```

$T(n) = T(n/2) + O(n)$, 因此平均复杂度为 $O(n)$ 。

Example 2.1.2. 如何求逆序对个数？利用归并排序代码。

对于一个序列 $a_1, a_2, \dots, a_n$, 若 $i < j$ 且 $a_i > a_j$, 则 (i, j) 就是一个逆序对。可以在归并排序的合并阶段顺带统计逆序对个数：合并时，如果左指针元素大于右指针元素，则右指针元素与左半部分剩余的所有元素构成逆序对。

2.2 计算机硬件

Note 2.2.1. 为什么实际过程中不常用归并排序而是用快速排序？归并排序需要额外的空间，而快速排序是原地排序。归并排序的时间复杂度为 $O(n \log n)$, 空间复杂度为 $O(n)$; 快速排序的平均时间复杂度为 $O(n \log n)$, 空间复杂度为 $O(\log n)$ (递归栈空间)。

Note 2.2.2. 常见存储单位之间的换算关系如下：8 bit = 1 byte, 1 KB = 1024 byte, 1 MB = 1024 KB, 1 GB = 1024 MB, 1 TB = 1024 GB, 1 PB = 1024 TB, 1 EB = 1024 PB。

个人电脑内存 4-8 GB, 服务器内存 16-64 GB, 云服务器内存 128 GB 以上。

Definition 2.2.1 (外部排序 (external sort)). 当数据量超过内存容量时，无法一次全部读入内存排序，于是分块读入、块内排序生成有序“归并段” (run)，再通过多路归并合成完整有序序列，是归并排序在磁盘上的推广。

当单机内存/磁盘都不够用时，需要把计算分发到多台机器，即**分布式程序**。典型框架：

$$\text{Hadoop} \begin{cases} \text{MapReduce} \\ \text{Spark} \end{cases}$$

MapReduce：把计算抽象为 Map（将输入映射为中间键值对）和 Reduce（按键聚合处理）两个阶段，由框架负责调度与容错，适合离线批处理。

Spark：基于内存的分布式计算引擎，利用 RDD（弹性分布式数据集）在内存中缓存中间结果，避免反复读写磁盘，因此对迭代式计算（如机器学习）远快于磁盘型的 MapReduce。

Note 2.2.3.

$$\text{主机} \begin{cases} \text{CPU} \\ \text{主存} \end{cases} \leftrightarrow_{\text{PCIe 显卡}} \begin{cases} \text{GPU} \\ \text{显存} \end{cases}$$

Definition 2.2.2 (译码器 (decoder)). 组合逻辑电路， n 位输入产生 2^n 条输出线且恰有一条为有效，用于把地址翻译成对应的存储单元选通信号；HBM 等内存内部依赖译码器完成行列寻址。

Definition 2.2.3 (HBM (High Bandwidth Memory, 高带宽内存)). 通过 TSV（硅通孔）垂直堆叠多层 DRAM 实现的高带宽内存，显著高于普通 DDR 内存的带宽且功耗更低，常用于 GPU 和 AI 加速卡（如 HBM2/HBM3）。

Note 2.2.4.

$$\text{CPU} \begin{cases} \text{intel(INTC)}(\text{指令, 优化}) \\ \text{AMD(AMD)} \end{cases} \begin{cases} \text{家用u9,u7,u5,u3} \\ \text{服务器Xeon} \end{cases} \begin{cases} \text{核数/线程数:8,16,96} \\ \text{主频:2.5GHz,3.5GHz} \\ \text{缓存 (Cache)} \end{cases}$$

Cerebras(CBRS),TBM

$$\text{存储器} \begin{cases} \text{三星} \\ \text{海力士 (SKHY)} \\ \text{美光 (MU)} \end{cases} \begin{cases} \text{容量:8GB,16GB} \\ \text{频率:DDR4,DDR5,4000MHz,8000MHz} \\ \text{时序: 颗粒,A-die,B-die,C-die} \end{cases}$$

套条 (8GB*2,16GB*2) 优先插 (2,4) 槽

$$\text{显卡} \begin{cases} \text{英伟达 (NVDA)(MKL 库) GTX} \rightarrow \text{RTX, DLSS} \\ \text{AMD} \end{cases}$$

Note 2.2.5. 软路由 openwrt AutoDL

3 数学补充 (DAY 3)

$$\text{答案} \begin{cases} \text{判别问题 (decision)} \\ \text{优化问题 (optimization)} \end{cases}$$

Example 3.0.1.

$$\text{判别问题} \begin{cases} \text{NP\&P} \\ \text{NPC} \\ \text{NP-hard} \end{cases}$$

N:non-deterministic (非确定性), P:Polynomial-time (多项式时间), C:Complete (完全)。

Definition 3.0.1 (P 类). 能在多项式时间内被确定性算法判定的判别问题集合。

Definition 3.0.2 (NP 类). 答案能被多项式时间内验证的问题集合; 等价地, 非确定性图灵机可在多项式时间内判定。显然 $P \subseteq NP$, $P = NP$ 是否成立是计算复杂性理论的核心未解问题。

Definition 3.0.3 (NP-hard). 若 NP 中所有问题都能多项式时间归约到问题 Q , 则 Q 是 NP-hard——它至少和 NP 中任何问题一样难, 但不要求自身属于 NP。

Definition 3.0.4 (NPC (NP 完全)). 既属于 NP 又是 NP-hard 的问题。NPC 是 NP 中最”难”的一类: 只要其中一个有多项式时间算法, 则 $P = NP$ 。

Example 3.0.2. Karp 21 个 NPC 问题: 1972 年 Karp 证明了包括 3-SAT、团、顶点覆盖、哈密顿回路、背包等在内的 21 个问题都是 NP 完全的, 奠定了 NP 完全性理论的基础。

3-SAT: 给定一个合取范式 (CNF) 公式, 其中每个子句恰好含 3 个文字, 问是否存在一组真值赋值使公式为真。3-SAT 是最经典的 NP 完全问题之一, 大量 NPC 证明都从它出发做归约。

旅行商问题 (TSP): 见下节组合优化中的定义, 是经典的 NP-hard 优化问题。

Definition 3.0.5 (哈密顿回路问题). 给定一个无向图, 判断是否存在一条经过每个顶点恰好一次的回路。

Definition 3.0.6 (欧拉路径问题). 给定一个无向图, 判断是否存在一条经过每条边恰好一次的路径。判定准则: 奇数度的点个数为 0 或 2。当奇数度的点个数为 2 时, 这 2 个奇数度的点为欧拉路径的起点和终点。

Definition 3.0.7 (旅行商问题). 给定一个完全图, 每条边都有一个权重 (距离), 要求找到一条最短的哈密顿回路。TSP 是 NP-hard 问题, 没有已知的多项式时间算法可以解决所有实例。常用的近似算法包括贪心算法、动态规划、分支定界法和启发式算法 (如遗传算法、模拟退火等)。

Example 3.0.3. 图 $\mathcal{G} = (\mathcal{V}, \mathcal{E})$, $\mathcal{V}$ 为点 (vertex) 集, $\mathcal{E}$ 为边 (edge) 集。上述三个问题是图论中最经典的路径/回路问题, 其中哈密顿回路与 TSP 是 NP-hard, 欧拉路径则存在多项式时间算法。

3.1 组合优化

$$\text{优化问题} \begin{cases} \text{变量} \\ \text{目标函数} \\ \text{约束条件} \end{cases} \begin{cases} \text{连续优化} \\ \text{组合优化} \\ \text{泛函优化} \end{cases} \begin{cases} \text{微分} \\ \text{差分} \\ \text{变分} \end{cases}$$

Example 3.1.1.

$$\begin{aligned} \min \quad & f(x) \\ \text{s.t.} \quad & f_i(x) \leq 0, \quad i = 1, \dots, m \\ & h_j(x) = 0, \quad j = 1, \dots, p \end{aligned}$$

Example 3.1.2. x 是一个排列

$$\begin{aligned} \min \quad & \sum_{i=1}^{n-1} w(x_i, x_{i+1}) + w(x_n, x_1) \\ \text{s.t.} \quad & (x_i, x_{i+1}) \in \mathcal{E} \\ & (x_n, x_1) \in \mathcal{E} \end{aligned}$$

Definition 3.1.1 (近似算法 (approximation algorithm)). 在多项式时间内给出与最优解相差有界的可行解, 用近似比衡量质量, 例如 TSP 的 MST 2-近似算法、欧氏 TSP 的 Christofides 算法 (近似比 $3/2$)。

Definition 3.1.2 (启发式算法 (heuristic algorithm)). 依靠经验规则快速找到可行解但不保证最优 (如贪心), 实现简单、速度快。

Definition 3.1.3 (仿生算法 (bio-inspired / metaheuristic)). 受自然过程启发的通用搜索框架, 如遗传算法 (GA)、粒子群优化 (PSO)、蚁群算法 (ACO)、模拟退火 (SA) 等, 常用于大规模组合优化求近似最优。

Definition 3.1.4 (OPT 下界). 能够多项式时间计算且不超过真实最优值 OPT 的界, 用于评价近似算法的性能。例如最小生成树权值就是欧氏 TSP 的一个有效下界。

Definition 3.1.5 (最小生成树 (MST)). 在无向带权连通图中选出 $n - 1$ 条边连通所有顶点且总权最小的树。Kruskal (贪心排序 + 并查集) 伪代码如下:

```
function kruskal(V, edges):
    sort edges by weight ascending
    DSU = each vertex its own set
    T = []
    for (u, v, w) in sorted edges:
        if find(u) != find(v):
            union(u, v); T.append((u, v, w))
        if |T| == |V| - 1: break
    return T
```

Prim 算法用堆每次取当前割边中权最小的边扩展生成树。两者复杂度均为 $O(m \log n)$ 。

Definition 3.1.6 (最大流问题). 给定有向图、源点 s 、汇点 t 及边容量, 求从 s 到 t 能输送的最大流量, 等价于最小割 (最大流最小割定理); 常用 Ford-Fulkerson、Edmonds-Karp、Dinic 算法求解。Ford-Fulkerson 伪代码如下:

```
function max_flow(G, s, t):
    f = 0
    while exists an augmenting path P in residual graph:
```

```

    c_f = min residual capacity on P
    augment f along P by c_f
    update residual capacities
return f

```

Theorem 3.1.1 (最大流最小割定理). 在有向网络 (V, E, c) 中, 从 s 到 t 的最大流等于 (s, t) -割的最小容量。

证明. 一方面, 任意可行流 f 穿过任意 (s, t) -割 (S, T) 时, 由流量守恒与容量约束知 $|f| = f(S, T) \leq c(S, T)$, 故最大流 $\leq$ 最小割。另一方面, Ford-Fulkerson 终止时残量网络中没有增广路, 令 S 为残量网络中从 s 可达的顶点集, 则 $s \in S, t \notin S$, 且跨越 (S, T) 的原边全部饱和、反向边全为 0, 故 $|f| = c(S, T)$ 。两方向结合得等式。□

Note 3.1.1. 图 $\mathcal{G} = (\mathcal{V}, \mathcal{E})$, 点集 $\mathcal{V} = \{\mathcal{V}_1, \dots, \mathcal{V}_n\}$, 边集 $\mathcal{E} = \{(u, v) \mid u, v \in \mathcal{V}\}$, 边权 $w(u, v)$ 。

Definition 3.1.7 (度 (degree)). 与顶点关联的边数, 记为 $\deg(v)$; 无向图中所有顶点度之和等于边数的两倍。

Definition 3.1.8 (邻居 (neighbor)). 与 u 有边直接相连的顶点集合 $N(u) = \{v \mid (u, v) \in \mathcal{E}\}$ 。

Definition 3.1.9 (子图与生成子图). **子图**: 顶点和边都是原图子集的图; **生成子图**: 顶点集等于原图顶点集的子图。

Definition 3.1.10 (连通性与连通分量). **连通性**: 无向图任意两点有路径则连通; 有向图分为**强连通** (任意两点互相可达)、**弱连通** (忽略方向后连通) 和**半连通** (任意两点至少一方可达)。**连通分量 (极大连通子图)**: 加上任意一个外部顶点都不再连通的连通子图, 即无法再加入任何顶点仍保持连通的子图。

Definition 3.1.11 (树). 连通且无环的无向图, n 个点的树一定有 $n - 1$ 条边, 任意两点间有唯一路径; **最小生成树**见上文。

为什么最小生成树可以成为 TSP 的一个很好的下界? TSP 最优回路去掉任意一条边后恰是一棵生成树, 因此

$$\text{OPT}_{\text{TSP}} > \text{OPT}_{\text{MST}} > \text{OPT}_{\min\{n-1 \text{ 条边}\}},$$

其中 OPT_{MST} 可由多项式时间算法精确求出, 从而给出 OPT_{TSP} 的一个有效下界。

4 模型与问题的差距 (DAY4)

4.1 图论

Note 4.1.1.

$$\text{树} \begin{cases} \text{链} \\ \text{星} \end{cases} \rightarrow \text{直径}$$

Example 4.1.1. 求树的最大直径?

$$\begin{aligned} \max \quad & \{\min d(u, v)\} \\ \text{s.t.} \quad & u, v \in \mathcal{V} \end{aligned}$$

思路：树的直径定义为所有点对距离的最大值。经典做法是两次 DFS/BFS：任选一点 s 出发找最远的点 u ，再从 u 出发找最远的点 v ，则 u, v 的距离就是直径。正确性基于”任意点到直径端点 u 的距离是到整棵树最远的距离”这一性质，用反证法可证。

$$\begin{aligned} \min \quad & \left\{ \max_{v \neq u, v \in \mathcal{V}} d(u, v) \right\} \\ \text{s.t.} \quad & u \in \mathcal{V} \end{aligned}$$

这是求图中”最小偏心距”（图的半径对应的中心），可用多源最短路或 Floyd 求解。

Note 4.1.2. 问题可行性与评价标准 benchmark?

Definition 4.1.1 (最近公共祖先 (LCA, Lowest Common Ancestor)). 在一棵有根树中，两个节点 u, v 的所有公共祖先里深度最大的那个。经典求法：

- **倍增法：**预处理每个节点的 2^k 级祖先，先让 u, v 对齐深度，再一起向上跳， $O(n \log n)$ 预处理、 $O(\log n)$ 每次查询；
- **Tarjan 离线算法：**借助 DFS 和并查集一次遍历回答全部询问， $O(n + \alpha(n))$ ；
- 转化为欧拉序 + RMQ：用 ST 表支持 $O(1)$ 查询。

倍增法伪代码如下：

```

// 预处理: up[u][k] = u 的第  $2^k$  级祖先, depth[u] = u 的深度
function lca(u, v):
    if depth[u] < depth[v]: swap(u, v)    // 对齐深度
    lift u up by (depth[u] - depth[v]) using up[][]
    if u == v: return u
    for k = log n down to 0:
        if up[u][k] != up[v][k]:
            u = up[u][k]; v = up[v][k]
    return up[u][0]

```

Note 4.1.3. 一个无向无环图是一棵树。

Note 4.1.4. 图的路径: 最短、最长、次长路。

Definition 4.1.2 (匹配 (matching)). 两两不相邻 (无公共顶点) 的边集。

Definition 4.1.3 (最大匹配). 边数最多的匹配; 若匹配边覆盖所有顶点则称为**完美匹配**。

Definition 4.1.4 (匈牙利算法 (Hungarian algorithm)). 求解二分图最大匹配的经典算法, 核心是不断寻找增广路并反转, 直至找不到增广路; 复杂度 $O(nm)$ (n 为点数, m 为边数)。
伪代码如下:

```

function hungarian(G):                                // G 为二分图
    match[v] = -1 for all right-side v
    for u in left side:
        visited = { }
        if not try_augment(u): break                  // 找不到增广路
    return number of matched edges

function try_augment(u):
    for each v adjacent to u:
        if v not visited:
            visited.insert(v)
            if match[v] == -1 or try_augment(match[v]):
                match[v] = u; return True
    return False

```

Definition 4.1.5 (点覆盖 (vertex cover)). 顶点子集, 使每条边至少有一个端点被选入;
最小覆盖: 顶点数最少的点覆盖。

Theorem 4.1.1 (König 定理). 在二分图中, 最大匹配大小等于最小点覆盖大小。

证明. 记最大匹配为 M 。先证 $|M| \leq$ 任意点覆盖的大小: 每条匹配边都必须被覆盖, 而匹配边两两不相邻, 故至少需 $|M|$ 个点, 所以最小点覆盖 $\geq |M|$ 。再证存在大小为 $|M|$ 的点覆盖: 从左侧未匹配点出发走”匹配边-非匹配边交替路”, 记交替路上可达的左侧点集为 L' 、可达的右侧点集为 R ; 令 $C = (L \setminus L') \cup R$ 。可验证 C 是点覆盖且 $|C| = |M|$ (这正是匈牙利算法终止时留下的结构), 故最小点覆盖 $\leq |M|$ 。两方面合起来即得等式。 $\square$

参考: oi-wiki.org

Lemma 4.1.1 (Burnside 引理). 设有限群 G 作用在集合 X 上, 则 X 在 G 下的轨道数等于

$$\frac{1}{|G|} \sum_{g \in G} |X^g|,$$

其中 $X^g = \{x \in X \mid g(x) = x\}$ 为 g 的不动点集合。

证明. 双计数集合 $A = \{(g, x) \mid g(x) = x\}$ 。按第一个分量计数得 $|A| = \sum_{g \in G} |X^g|$; 按第二个分量计数, 对每个轨道 O 中的 x , 满足 $g(x) = x$ 的 g 恰构成一个陪集, 个数均为 $|G|/|O|$, 故每个轨道贡献 $|O| \cdot |G|/|O| = |G|$ 个对。因此 $|A| = (\text{轨道数}) \cdot |G|$, 整理即得结论。 $\square$

Theorem 4.1.2 (Pólya 计数定理). 用 k 种颜色给 n 个位置染色, 在置换群 G 作用下本质不同的染色数为

$$\frac{1}{|G|} \sum_{g \in G} k^{c(g)},$$

其中 $c(g)$ 是置换 g 中循环的个数。

证明. 由 Burnside 引理, 轨道数 $= \frac{1}{|G|} \sum_{g \in G} |\text{Fix}(g)|$ 。一个染色在 g 下不变的充要条件是其每个循环内的位置颜色相同: 每个循环有 k 种选色, 且不同循环独立, 故 $|\text{Fix}(g)| = k^{c(g)}$ 。代入即得。 $\square$

Definition 4.1.6 (团 (clique)). 任意两点之间都有边相连的顶点子集 (完全子图); 最大团问题是 NP-hard 的。

Definition 4.1.7 (染色问题). 给图的每个顶点染色, 使相邻顶点颜色不同, 所用最少颜色数称为**色数**; 判定图是否 3-可染色是 NP 完全问题。

Example 4.1.2. 有一个 4×4 的矩阵, 每个位置上都有一个灯泡。按下某个位置时, 会改变它上下左右四个位置的开关状态。对任意给定的一种初始状态, 问最少需要多少次操作才能使所有灯泡全部熄灭?

思路 (熄灯问题 / Lights Out): 设 $x_{ij} \in \{0, 1\}$ 表示是否按下位置 (i, j) , 则每个位置的最终状态由初始状态与它自身及上下左右共 ≤ 5 个按钮的异或和决定。要求最终全灭, 得到一个 $\mathbb{F}_2$ 上的线性方程组 $A\mathbf{a} = \mathbf{b}$ ($\mathbf{b}$ 由初始状态决定), 用高斯消元解出 $\mathbf{a}$ 。由于按两次等于没按, 最少次数正是解中 1 的个数; 若解不唯一, 取 $\sum x_{ij}$ 最小的那组。

有一个 $m \times n$ 的矩阵, 每个位置上可能存在一个怪物。可以选择某一行或某一列进行一次轰炸, 问最少需要多少次才能消灭全部怪物?

思路 (二分图最小点覆盖): 把每一行与每一列分别作为二分图两侧的顶点, 每个位于 (i, j) 的怪物作为连接“第 i 行”与“第 j 列”的边。轰炸某行/列等价于在对应顶点中选点, 目标是让每条边 (怪物) 至少被选中一次的一个端点覆盖——这正是**最小点覆盖**问题。由 König 定理, 二分图中最小点覆盖大小等于最大匹配大小, 故用匈牙利算法求最大匹配即可得到最少轰炸次数。

4.2 统计

Definition 4.2.1 (二项分布 (Binomial distribution)). n 次独立重复试验 (每次成功概率为 p) 中成功次数 X 服从参数 n, p 的二项分布 $X \sim B(n, p)$, 其分布列为

$$P(X = k) = \binom{n}{k} p^k (1-p)^{n-k}, \quad 0 \leq k \leq n,$$

期望 $\mathbb{E}[X] = np$, 方差 $\text{Var}(X) = np(1-p)$ 。

Definition 4.2.2 (泊松分布 (Poisson distribution)). 单位时间或单位区域内随机事件发生次数 X 服从参数 λ 的泊松分布 $X \sim \text{Poi}(\lambda)$, 其分布列为

$$P(X = k) = \frac{\lambda^k e^{-\lambda}}{k!}, \quad k = 0, 1, 2, \dots,$$

期望与方差均为 λ 。它是 n 很大、 p 很小而 $np = \lambda$ 保持恒定时的二项分布极限 (泊松近似)。

Definition 4.2.3 (指数分布 (Exponential distribution)). 连续型随机变量 X 的密度函数为

$$f(x) = \lambda e^{-\lambda x}, \quad x \geq 0,$$

记为 $X \sim \text{Exp}(\lambda)$, 期望 $\mathbb{E}[X] = 1/\lambda$, 方差 $\text{Var}(X) = 1/\lambda^2$, 具有无记忆性 $P(X > s + t \mid X > s) = P(X > t)$ 。它是泊松过程两次事件之间的等待时间的分布。